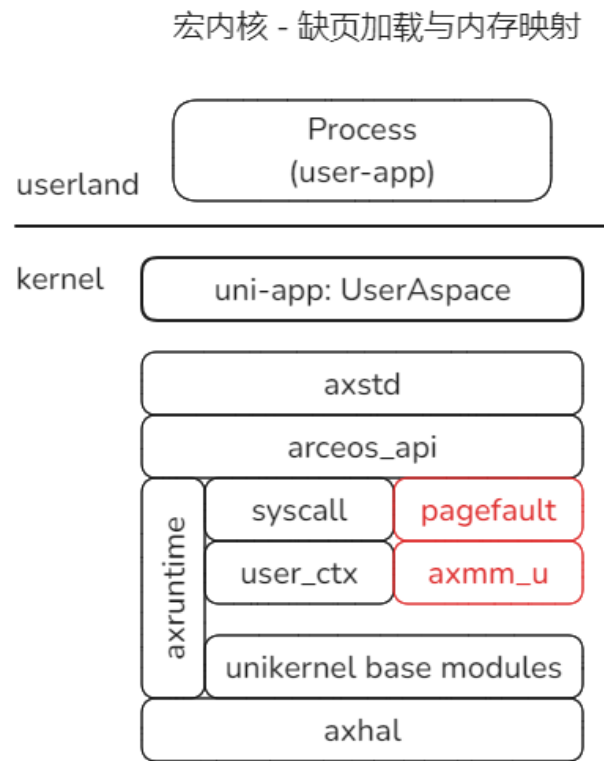
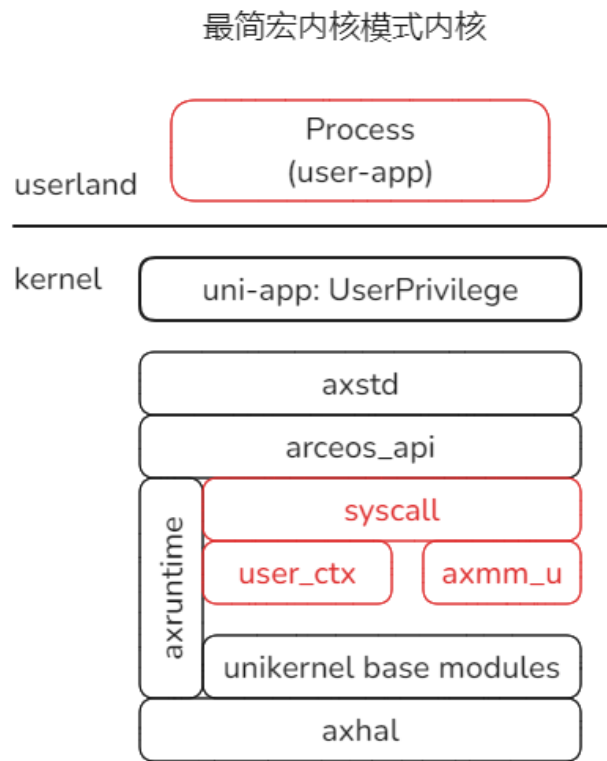


秋冬季训练营三阶段 组件化内核设计与实践(5)

清华大学 & 山东乾云启创
泉城实验室安全操作系统联合创新中心
石磊

2024.11.20

M.2.0 UserAspace



本节目标:

1. 缺页加载的原理和机制
2. 内存映射相关的系统调用

实验命令行:

`make payload`

`./update_disk.sh payload/origin/origin`

`make run A=tour/m_2_0 BLK=y`

从实验m_1_0到m_2_0

按照如下方式，修改m_1_0关于初始化用户栈的代码，从直接映射改为Lazy映射。

```
- let ustack_top = init_user_stack(&mut uspace, true).unwrap();  
+ let ustack_top = init_user_stack(&mut uspace, false).unwrap();
```

屏幕提示未处理的用户页异常：

```
[ 0.391718 0:4 axhal::arch::riscv::trap:24] No registered handler for trap PAGE_FAULT  
[ 0.394909 0:4 axruntime::lang_items:5] panicked at modules/axhal/src/arch/riscv/trap.rs:25:9:  
Unhandled User Page Fault @ 0x1002, fault_vaddr=VA:0x3fffffffffc (WRITE | USER):
```

参考用户应用origin的实现（汇编语言）：

```
"addi sp, sp, -4",  
"sw a0, (sp)",
```

得知确实是第二行代码向栈的[基址地址-4]位置写入导致的缺页写异常。

对照m_2_0的代码，相对于m_1_0增加了缺页异常处理：

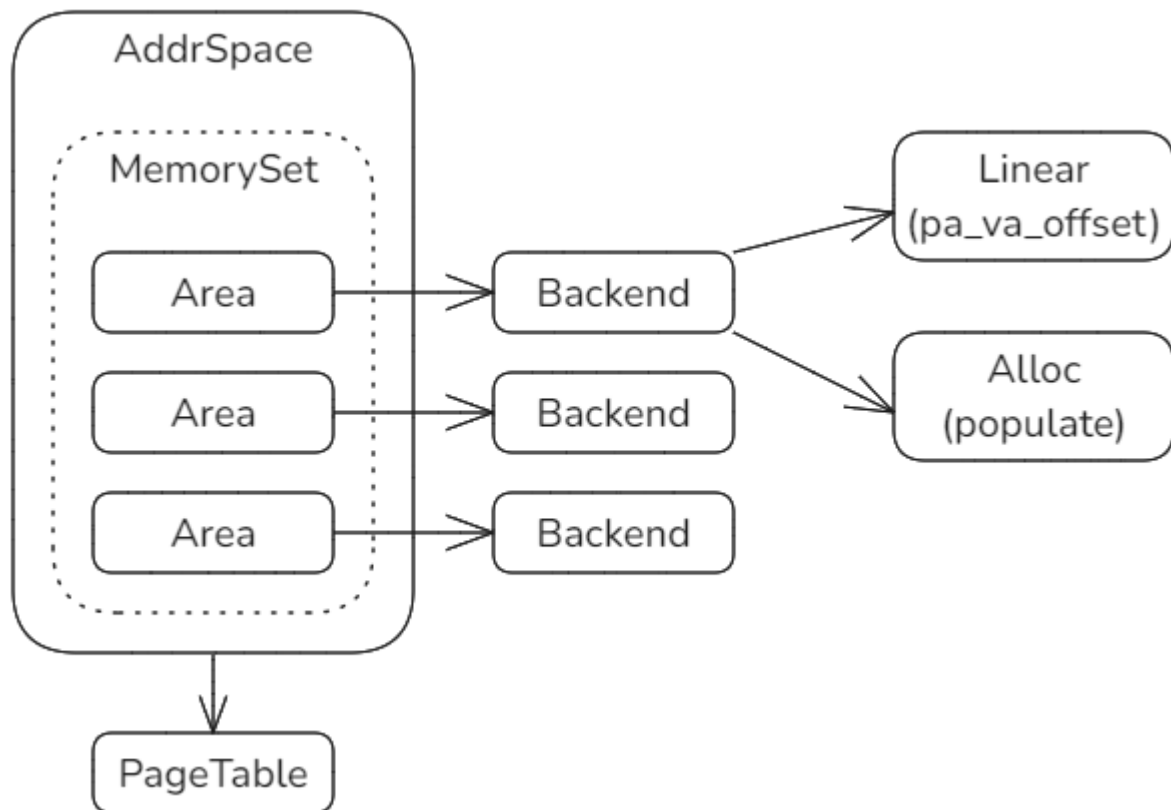
```
#[register_trap_handler(PAGE_FAULT)]  
fn handle_page_fault(...) -> bool {  
    aspace.XXX.handle_page_fault(...) {  
    }  
}
```

发送缺页异常时，由aspace的handle_page_fault来完成对物理页帧的申请与映射。

方法handle_page_fault是地址空间映射功能重要触发入口，以下以它为线索说明地址空间映射方面的功能。

宏内核地址空间管理相关对象的层次构成

地址空间管理涉及的主要对象：AddrSpace，MemorySet，MemoryArea和Backend的两种实现。



AddrSpace: 包含一系列有序的区域并对应一个页表。

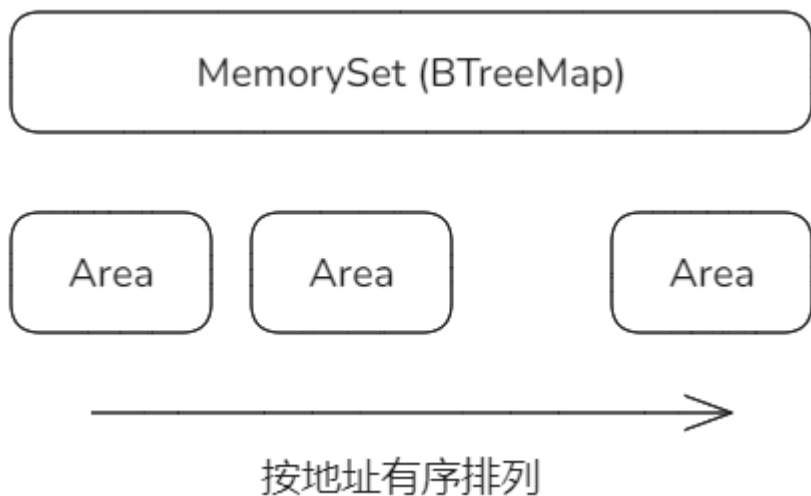
MemorySet: 对BTreeMap的简单封装，对空间下的各个MemoryArea进行有序管理。

MemoryArea: 对应一个连续的虚拟地址内存区域，关联一个负责具体映射操作的后端Backend。

Backend: 负责具体的映射操作，不同的区域MemoryArea可以对应不同的Backend。目前支持两种后端类型：**Linear**和**Alloc**。

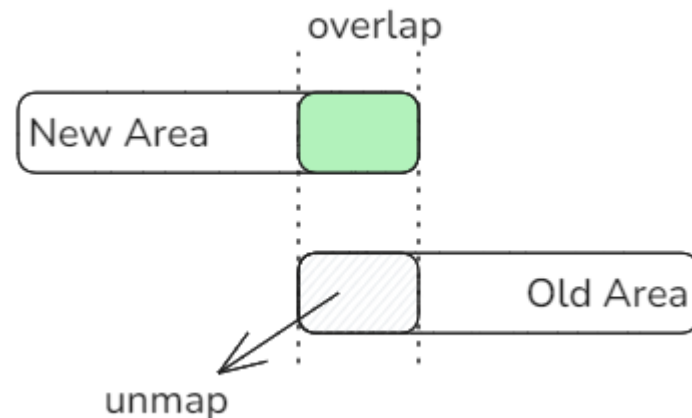
地址空间区域MemorySet

```
pub struct MemorySet<B: MappingBackend> {  
    areas: BTreeMap<B::Addr, MemoryArea<B>>,  
}
```

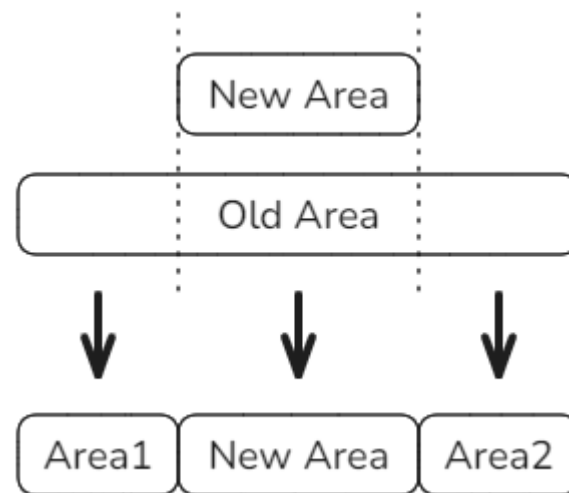


对地址空间的主要操作就是查找目标区域或者是查找区域之间的空隙，因此对它们的有序管理是保证性能的关键。

地址空间区域重叠时，原区域的重叠部分被 Unmap，然后新区域被映射到空间中。

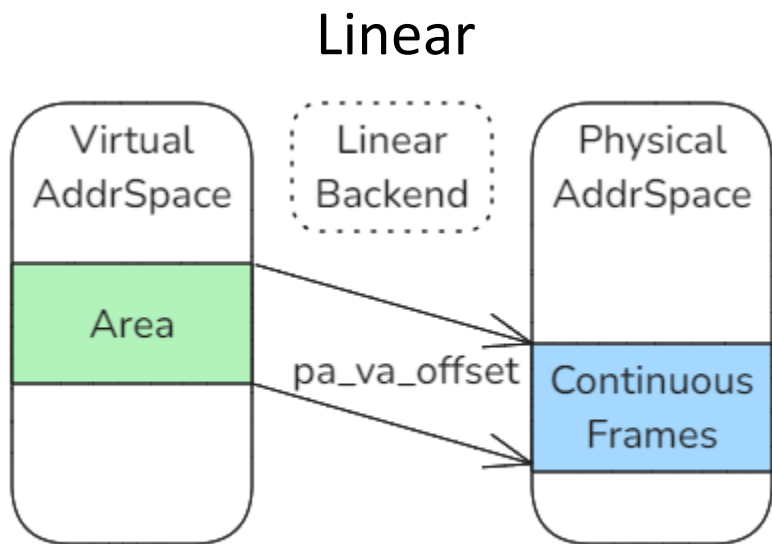


特殊情况：新区域位于旧区域的中间。



地址空间区域映射后端Backend

后端负责针对空间中特定区域的具体的映射操作， Backend从实现角度是一个Trait。

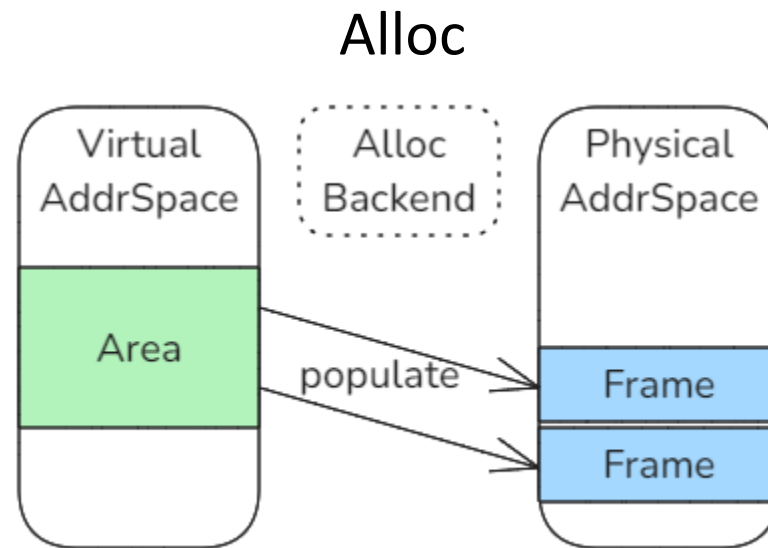


Linear的应用场景：

目标物理地址空间区域已经存在，直接建立映射关系。

可以用于设备MMIO区域映射以及特殊的共享地址区域映射等。

注意：对应的物理页帧**必须连续**。



Alloc的应用场景：

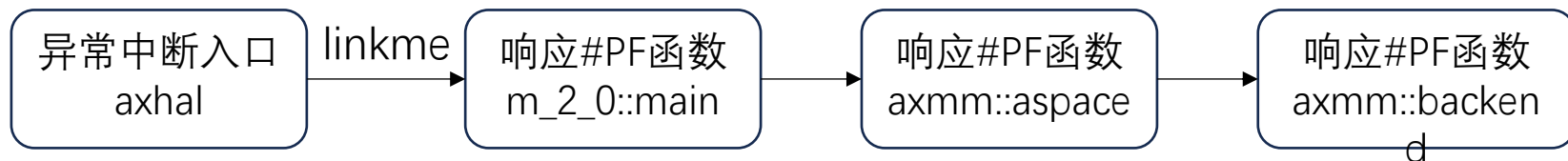
仅建立空映射，当真正被访问时将会触发缺页异常，然后在缺页响应函数内部完成物理页帧的申请和补齐映射。

也就是Lazy方式。

注意：按页映射，对应的物理页帧通常情况下**不连续**。

缺页异常响应函数的实现

多级调用handle_page_fault，最终到backend::alloc中的实现。



modules/axmm/src/backend/mod.rs

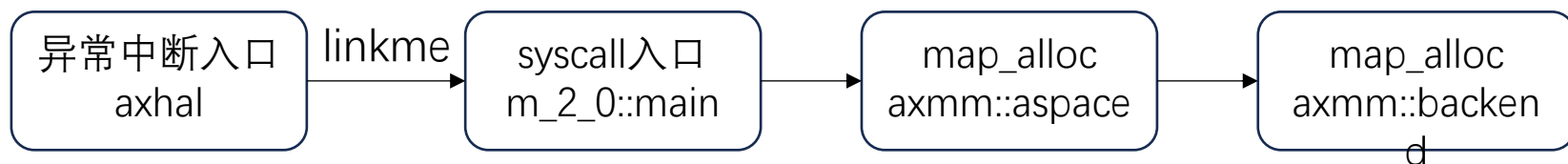
```
pub(crate) fn handle_page_fault_alloc(
    &self,
    vaddr: VirtAddr,
    orig_flags: MappingFlags,
    pt: &mut PageTable,
    populate: bool,
) -> bool {
    if populate {
        false // Populated mappings should not trigger
    } else if let Some(frame) = alloc_frame(true) {
        // Allocate a physical frame lazily and map
        // `vaddr` does not need to be aligned. It will be
        // aligned during `pt.remap` regardless of
        pt.remap(vaddr, frame, orig_flags)
            .map(|(_, tlb)| tlb.flush())
            .is_ok()
    } else {
        false
    }
}
```

第一步，申请物理页帧

第二步，在页表中完成映射。

系统调用sys_mmap的实现

实现方法与缺页异常处理的流程类似，触发原因不同。sys_mmap由应用调用libc的某些方法触发。



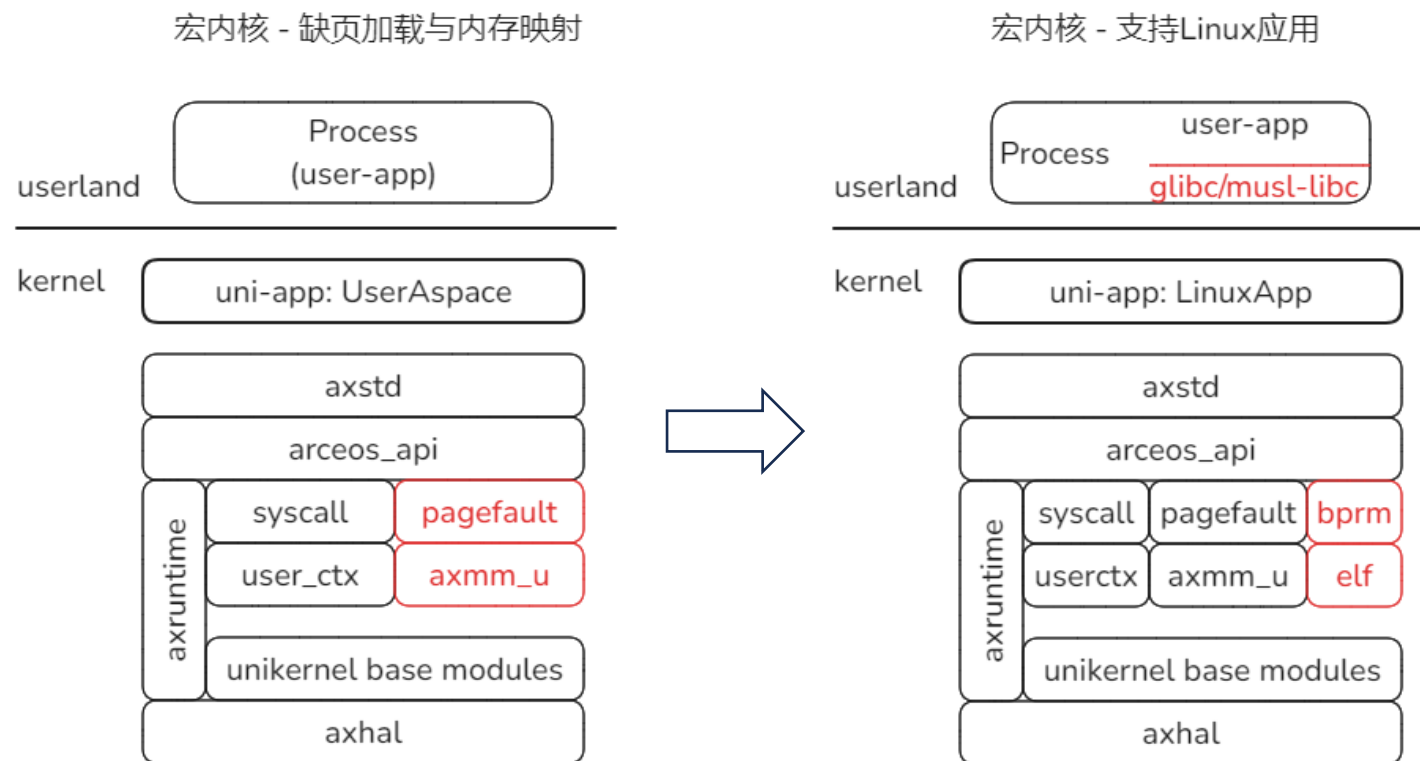
modules/axmm/src/backend/mod.rs

```
pub(crate) fn map_alloc(&self, start: VirtAddr, size: usize, flags: MappingFlags) {
    if populate {
        for addr in PageIter4K::new(start, start + size).unwrap() {
            if let Some(frame) = alloc_frame(true) {
                if let Ok(tlb) = pt.map(addr, frame, PageSize::Size4K, flags) {
                    tlb.ignore(); // TLB flush on map is unnecessary, as there is no fault
                } else {
                    return false;
                }
            }
        }
    }
    true
} else {
    let flags = MappingFlags::empty();
    pt.map_region(start, |_| 0.into(), size, flags, false, false)
        .map(|tlb| tlb.ignore())
        .is_ok()
}
```

populate为true时，
与缺页异常处理类似。
申请页帧后直接完成映射，
将来不会触发#PF。

populate为false时，
仅建立空映射，
将来访问时触发#PF。
然后进入缺页流程。

M.3.0 LinuxApp



本节目标:

1. glibc/musl-libc与内核的参数协同
2. 启动Linux原始应用hello

实验命令行:

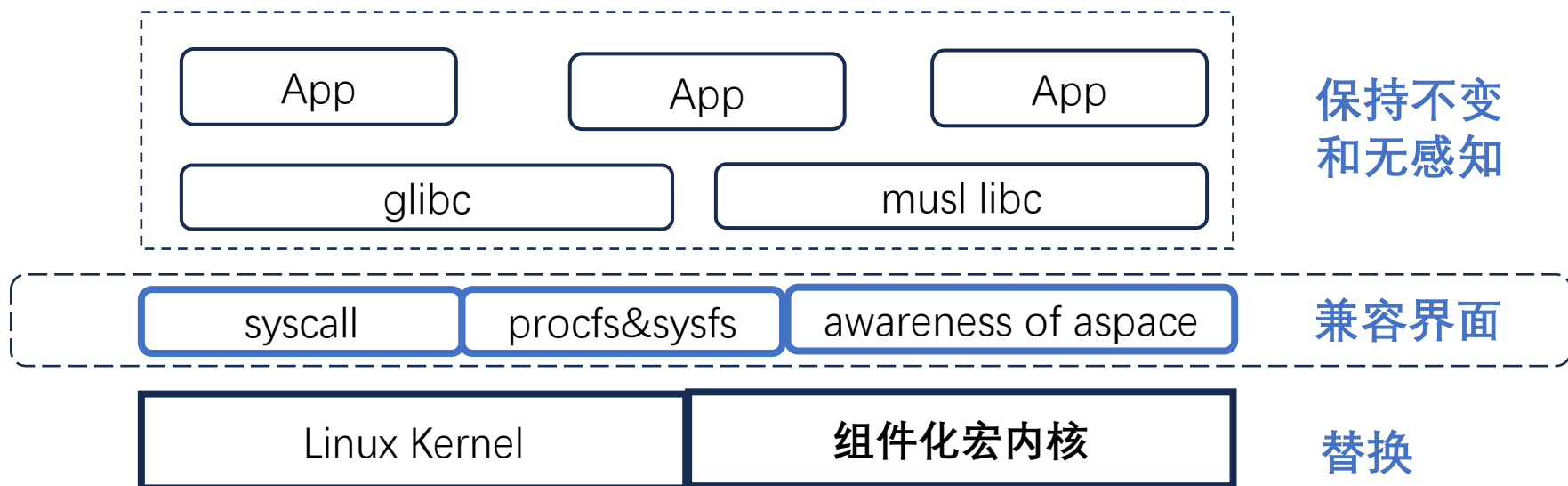
make payload

./update_disk.sh payload/hello_c/hello

make run A=tour/m_3_0 BLK=y

需要解决的问题

如何让Linux的原始应用（二进制）直接在我们的宏内核上直接运行？



在应用和内核交互界面上实现兼容。

兼容界面包含三类：

- 1) syscall
- 2) procfs & sysfs等伪文件系统
- 3) 应用、编译器和libc对地址空间的假定，涉及某些参数定义或某些特殊地址的引用

ELF格式应用的加载

多少应用在编译时都默认采取ELF格式，内核负责解析和加载各段到用户地址空间中的适当位置。
示例m_3_0使用的应用hello的格式：

```
cloud@server:~/gitWork/oscamp/arceos$ riscv64-linux-musl-readelf -lW ./payload/hello_c/hello

Elf file type is DYN (Position-Independent Executable file)
Entry point 0x50e
There are 5 program headers, starting at offset 64

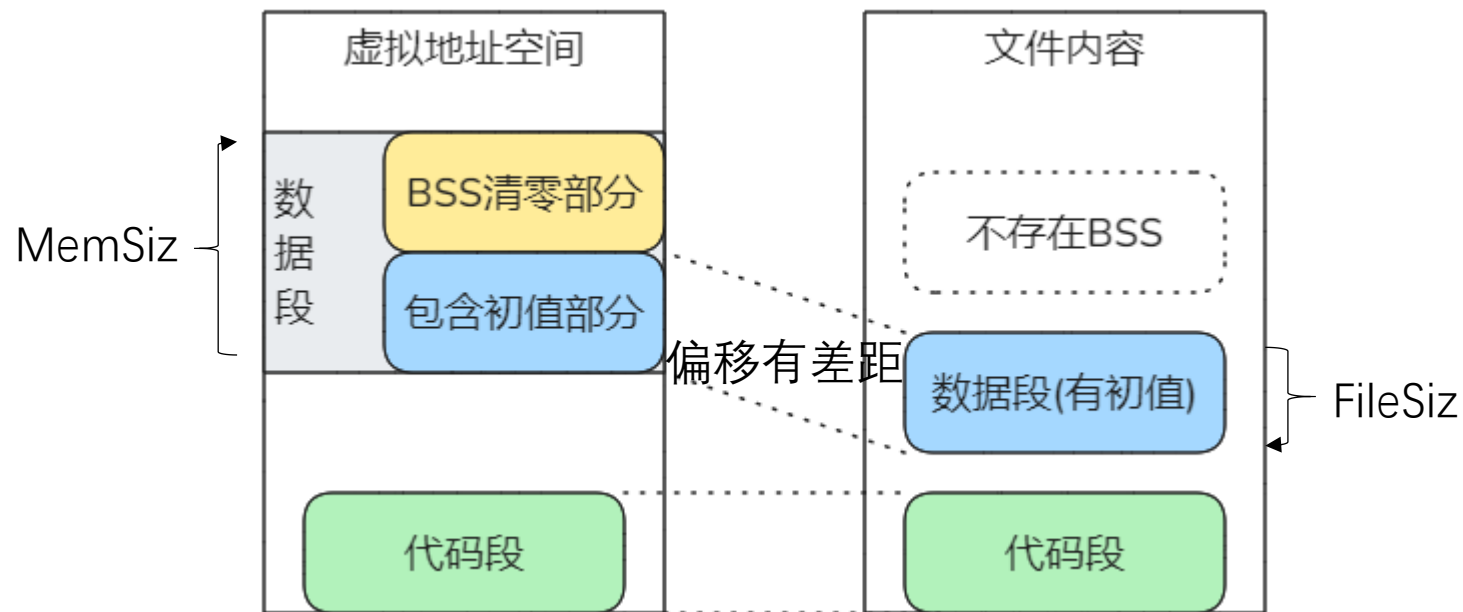
Program Headers:
Type           Offset      VirtAddr           PhysAddr          FileSiz        MemSiz          Flg Align
LOAD           0x00000000 0x0000000000000000 0x0000000000000000 0x0017cc 0x0017cc  R E 0x1000
LOAD           0x001e70 0x00000000000002e70 0x00000000000002e70 0x000338 0x0009a8  RW 0x1000
DYNAMIC        0x001e90 0x00000000000002e90 0x00000000000002e90 0x000170 0x000170  RW 0x8
GNU_STACK      0x000000 0x0000000000000000 0x0000000000000000 0x000000 0x000000  RW 0x10
GNU_RELRO      0x001e70 0x00000000000002e70 0x00000000000002e70 0x000190 0x000190  R  0x1
```

需要获取的信息：

- 1. Entry point应用的入口地址
 - 2. 两个Type为LOAD的段，表示需要加载。分别是代码段和数据段，从Flag属性可以区分。
- 注意：数据段的文件内偏移Offset和虚拟地址VirtAddr不一样，且FileSiz和MemSiz不一样。

ELF格式应用的加载

需要注意文件内偏移和预定的虚拟内存空间内偏移可能不一致，特别是数据段部分。



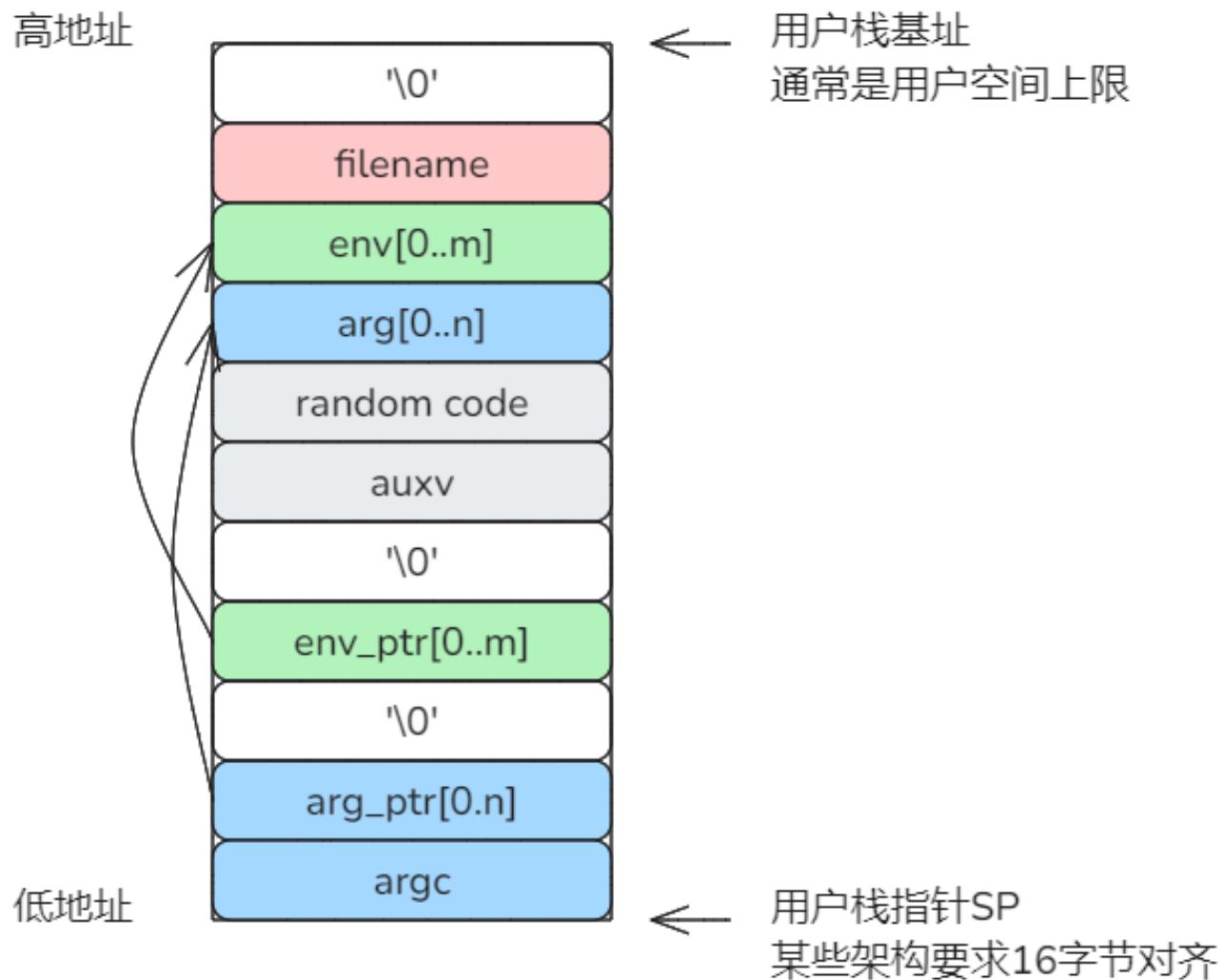
通常ELF为了节省空间，紧凑存储，从Offset和FileSiz定位段。

内核根据VirtAddr和MemSiz把段安置到目标虚拟内存位置。

由于BSS部分全零，所以ELF文件中只是标记位置和长度，不存实际数据，内核直接预留空间后**清零**。

应用的用户栈初始化

Linux应用基于glibc/musl-libc等库编译，libc在调用应用的main之前，检查用户栈上的参数等内容。而应用启动之后，也可能会调用这些参数。内核需要在切换到首应用前，为应用准备栈上内容。



应用main函数的原型:

```
int main(int argc, char *argv[], char *envp[]);
```

`argc`: 参数个数

`argv`: 参数指针数组，指向参数区。

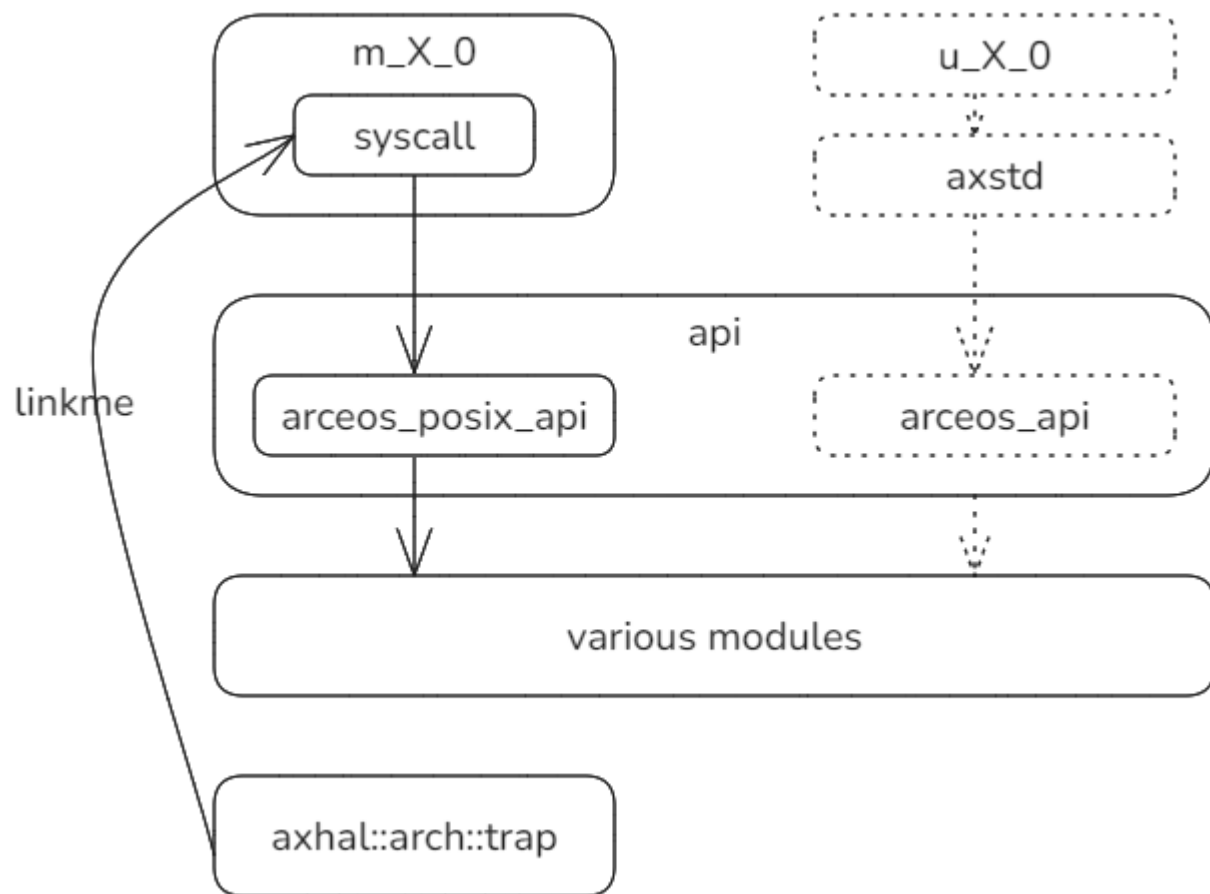
`envp`: 环境变量指针数组，指向环境变量区。

`auxv`是内核与工具链协作的高级形式，不设置或设置有错误，都会回退到低级形式。

内核在用户栈上按照此布局设置后，设置SP寄存器，再切换到用户态开始应用的执行。

支持系统调用的层次结构

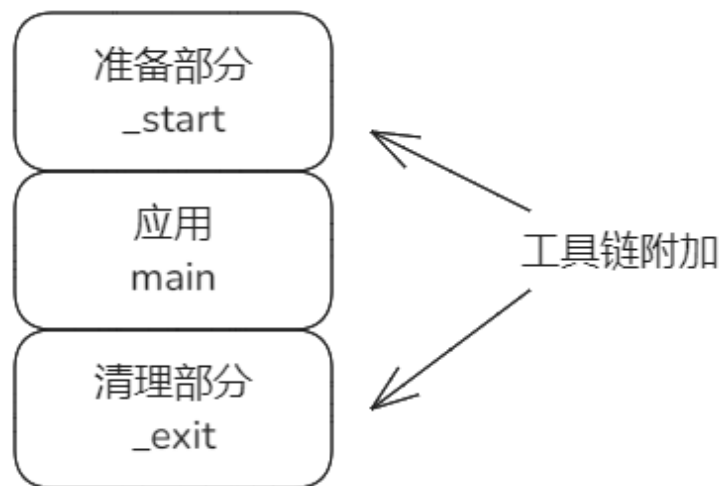
宏内核的系统调用实现路径大致与Unikernel并列。



`arceos_posix_api`是参照POSIX规范封装的接口， 主要提供对`syscall`的支持。

示例m_3_0涉及的系统调用

示例m_3_0基于musl工具链以静态方式编译，工具链为应用附加的部分也会调用syscall。



运行状况：

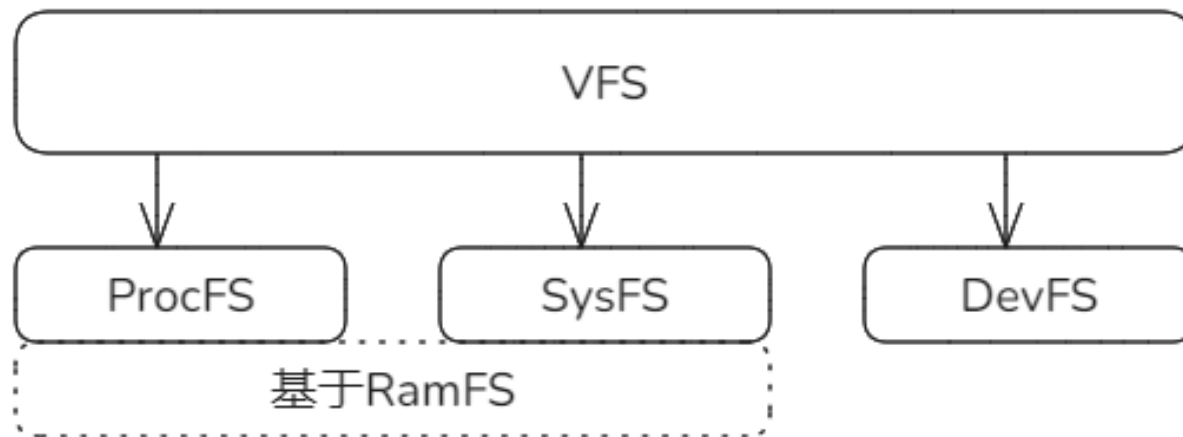
```
handle_syscall [96] ... 96 -> set_tid_address
handle_syscall [29] ... 29 -> ioctl
Unimplemented syscall: SYS_IOCTL
handle_syscall [66] ...
Hello, UserApp!
handle_syscall [66] ... 66 -> writev

handle_syscall [94] ... 94 -> exit_group
[SYS_EXIT_GROUP]: system is exiting ..
```

对于不同的体系结构，系统调用号不同。
示例是基于riscv64的系统调用号规范。

对Linux常用文件系统的支持

Linux常用的文件系统包括ProcFS、SysFS和DevFS，与普通文件系统不同，它们属于伪文件系统，具有相同的接口和抽象，但是Backend却不是普通的数据。



Procfs 用于提供内核和进程信息的接口。它通常挂载在 `/proc` 目录下，包含了大量关于系统和进程的信息。

Sysfs 用于向用户空间暴露设备信息。它通常挂载在 `/sys` 目录下。该文件系统主要用于替代传统的 `devfs`。

Devfs 用于向用户空间暴露设备和驱动信息。目前主要是为了兼容性而存在。

axfs_devfs组件

procfs和sysfs都是axfs_ramfs的实例

课后练习 - 实现mmap系统调用

[sys_map]: 为宏内核支持sys_mmap系统调用, 实现对文件的映射。

预备:

1. 如果没有**exercises/sys_map**或**mapfile_c**应用, 回到main分支执行**git pull** 更新工程, 再新建分支做练习
2. 执行如下三步:

make payload

./update_disk.sh payload/mapfile_c/mapfile

make run A=exercises/sys_map/ BLK=y

```
handle_syscall [222] ...
```

```
[ 0.435143 0:4 axruntime::lang_items:5] panicked at exercises/sys_map/src/syscall.rs:143:5:  
not implemented: no sys_mmap!
```

要求:

1. 实现sys_mmap系统调用, 使测例通过。
2. 再次执行**make run A=exercises/sys_map/ BLK=y**

预期: 测例通过时, 显示右图的输出效果。

提示:

可以参考m_1_0中加载应用到用户地址空间的代码段。

```
handle_syscall [222] ...  
paddr: PA:0x80769000  
handle_syscall [66] ...  
Read back content: hello, arceos!  
handle_syscall [57] ...  
handle_syscall [66] ...  
MapFile ok!  
handle_syscall [94] ...  
[SYS_EXIT_GROUP]: system is exiting ..  
monolithic kernel exit [Some(0)] normally!  
cloud@server:~/gitWork/oscamp/arceos$
```

下节课（周五）由郑友捷老师讲解
组件化宏内核的实践经验。